

PROJECT AND TEAM INFORMATION

Project Title

VitalRoute: A Smart Multi-Specialty Hospital Management & Emergency Routing System

Student/Team Information

<i>Team Name:</i> <i>Team ID</i>	<i>Team VitalRoute</i> <i>DSCPP-III-2026-T120</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Agarwal, Nakshatra –</i> <i>2510370215</i> <i>2510370215@geu.ac.in</i> <i>Section: ML1</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Saini, Devesh –</i> <i>2510370251</i> <i>2510370251@geu.ac.in</i> <i>Section: ML4</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Sharma, Vasav –</i> <i>25110224030</i> <i>25110224030@geu.ac.in</i> <i>Section: E</i> 

PROPOSAL DESCRIPTION

Motivation

Every day, multi-specialty hospitals have to make decisions that genuinely cannot wait: who gets treated first, whether an ICU bed is actually open right now, and how much a patient owes at discharge. In smaller clinics and mid-sized hospitals, these decisions are often still handled the old way, using paper registers, phone calls between wards, and staff physically running down hallways to check on beds. That might work on a slow day, but during an emergency, every minute spent searching for an available operation theatre (OT) could change a critical case's outcome.

There is also a fairness issue hiding in plain sight. A patient walking in with a sprained ankle and someone having a cardiac emergency should not be treated based on who arrived first. Yet a surprising number of basic hospital systems default to exactly that: first-come, first-served.

We chose this problem because it gives us a realistic, cohesive scenario to apply everything we are learning this semester. Instead of stitching together unrelated textbook exercises, we can use priority queues for emergency triage, graphs for finding the closest ICU bed, and object-oriented programming to model staff and billing. It is a chance to build something that actually solves a real-world logic problem.

State of the Art/Current solution

Large hospital networks already run on commercial platforms known as Hospital Management Information Systems, or HMIS, along with database heavy Electronic Health Record tools. Smaller nursing homes usually cannot afford these systems, so they end up relying on fragmented spreadsheets or manual paper registers instead. Even hospitals that do have basic software often still handle emergency triage and bed allocation through human judgment rather than automated, severity aware routing.

We are not trying to build a massive commercial product to compete with these platforms. Our goal is narrower and more technical than that. We want to simulate the core decision making that these systems are supposed to handle, and build a lightweight, severity aware simulator from scratch using fundamental data structures and clean object oriented design.

Project Goals and Milestones

Our goal is to build a working C++ console simulator that handles patient intake, intelligent emergency routing, persistent medical records, and automated billing.

Here is how we are mapping hospital operations onto our course syllabus. Patients, staff, and billing will be built as a clean class hierarchy using inheritance and polymorphism in C++. For triage and scheduling, priority queues make sure critical emergency patients jump to the front of the line, standard queues handle routine patient check ups, circular queues manage the daily, repeating Operation Theatre, or OT, schedule, and dequeues take care of ambulance and transfer requests so a genuinely urgent one can be pushed to the front while a routine one waits at the back.

For resource routing, we will represent hospital departments, ICUs, and wards as a graph and use shortest path logic to instantly find the closest available bed. Patient histories will be stored in doubly linked lists, indexed using hashing for instant lookups and organized in AVL trees so records can be searched within specific date ranges, and we will use binary search to quickly check doctor availability. Finally, we are building a billing engine that uses operator overloading and virtual functions to tally up automatic resource charges and manual additions, applying insurance deductions at the very end.

Project Timeline (weeks running through November 1, 2026):

Week	Dates	Milestone Target
1	Sep 2 – Sep 8	Finalize requirements, feature list, architecture, and Phase I proposal.
2	Sep 9 – Sep 15	Design core OOP class hierarchy (Patient, Doctor, Staff, Charge).
3	Sep 16 – Sep 22	Build Patient and Staff management, unique IDs, and binary-search doctor availability.
4	Sep 23 – Sep 29	Implement priority-queue emergency triage and graph-based bed routing.
5	Sep 30 – Oct 6	Build circular-queue OT scheduling and dequeue-based ambulance handling.
6	Oct 7 – Oct 13	Set up medical records (doubly linked lists, hashing indexes, AVL trees).
7	Oct 14 – Oct 20	Finalize the billing engine (operator overloading) and the C++ file persistence layer.
8	Oct 21 – Oct 27	Integrate all modules into the console interface; begin unit testing.
9	Oct 28 – Nov 1	Final bug fixes, polish documentation, and prepare for Phase II submission.

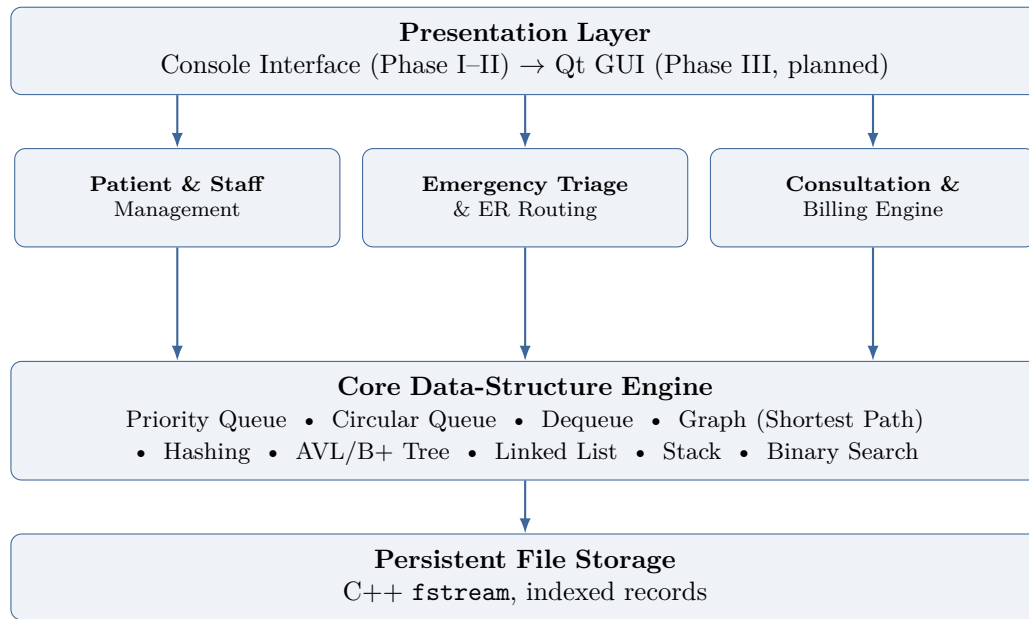
Project Approach

We are splitting the workload across two language layers to match our two courses. The C data-structure engine, the priority queues, graphs, hashing, trees, linked lists, and sorting routines, will be written in C, focusing heavily on pointers and manual memory management.

Sitting right on top of the C engine, the C++ object-oriented layer will use classes, virtual functions, exception handling, and STL containers to model the real-world hospital workflow.

We are taking a bottom-up approach. We will build and test the data structures first, wrap the C++ OOP layer around them, and finally stitch it all together with a user-facing console menu. We are using the standard GCC/G++ compiler, working in VS Code or Code::Blocks, and managing our collaboration through GitHub.

System Architecture (High Level Diagram)



The presentation layer can be replaced without affecting anything else. When the Phase III GUI is added, it will take the place of the console without requiring changes to any module, the engine, or the storage layer.

Project Outcome/Deliverables

By the end of the semester, we will submit:

- *A fully working, console-based hospital simulator utilizing severity-based triage and automated routing.*
- *A persistent file-storage system, ensuring patient records survive between program runs.*
- *Clean, documented C and C++ source code that actively utilizes every core topic from the syllabus.*
- *A technical document mapping every hospital feature to its underlying data structure or OOP concept.*
- *(Bonus/Stretch Goal): A basic Qt-based graphical interface and a role-based daily summary report generator.*

Assumptions

Since this is an educational simulator, we are setting a few boundaries. It will never connect to real hospital databases, payment gateways, or live insurance servers; all test data is mocked. We assume the system is running locally on a single machine (no concurrent terminals). Hospital layouts (beds, departments, OTs) and insurance policies are simplified, fixed inputs that we will configure ourselves.

References

1. *Reema Thareja, Data Structures Using C, Oxford University Press.*
2. *E. Balagurusamy, Object-Oriented Programming with C++, McGraw Hill Education.*
3. *cppreference.com – for C++ language and STL documentation.*
4. *GeeksforGeeks (geeksforgeeks.org) – for validating complex algorithm logic.*
5. *Qt Documentation (doc.qt.io) – for our planned Phase III GUI implementation.*